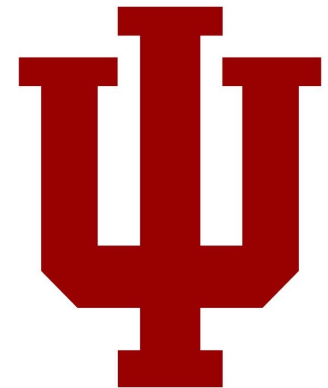


Exam Review

Engr 315: Hardware / Software Codesign

Andrew Lukefahr

Indiana University



Some material taken from EECS370 at U. of Michigan

Announcements

- P7:
 - AG due tonight.
 - Demo due tomorrow
- Exam: on Wednesday!
- P8: Due after exam...

Exam Details

- Main 5 sections
 - Multiple questions / section
- Some short answer
- Some fill-in-the blank/code/table

A “Cheat” Sheet is Allowed

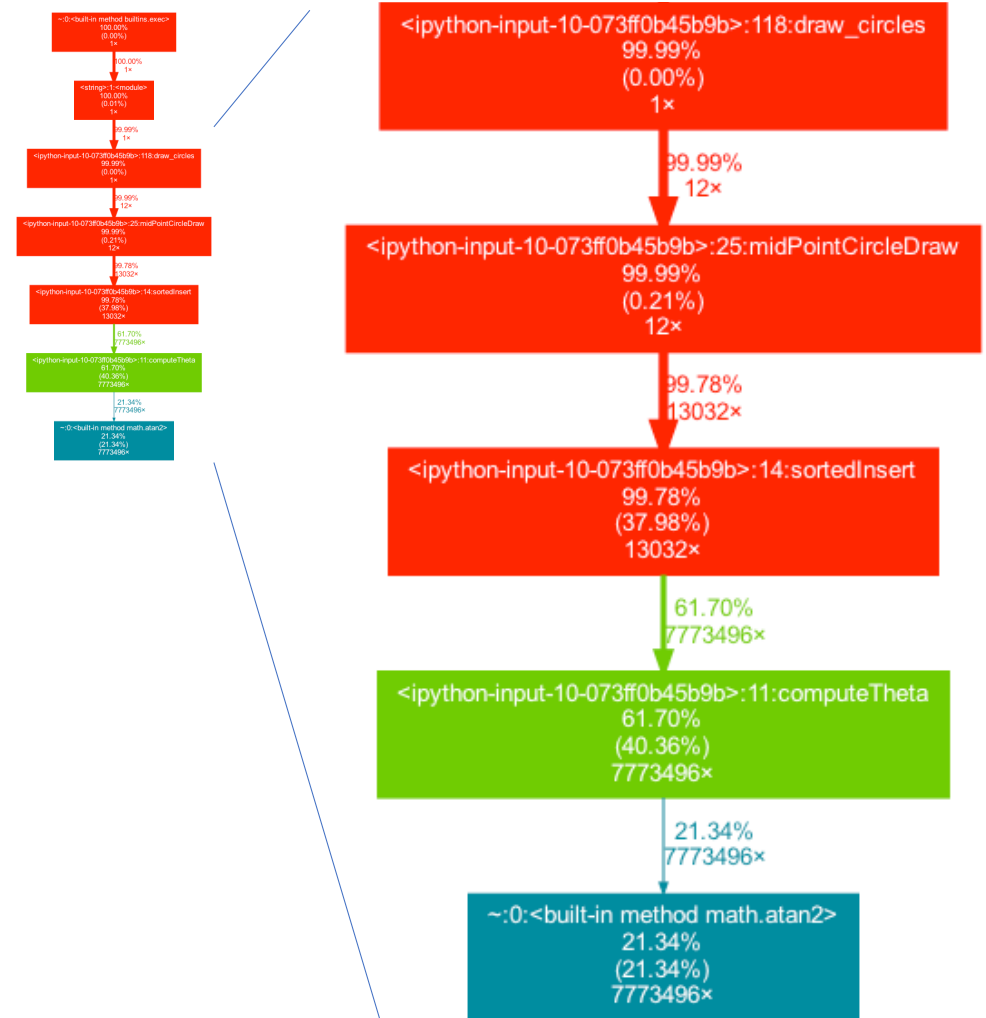
- 2-sided
- 8.5”x11” paper
- Handwritten (not photocopied)

Major Topics

- Performance Profiling
- Data Structures
- Bus Interfaces
- MMIO
- DMA
- Pipelining

Performance Profiling

- What is profiling?
- What function should we be optimizing here?
- How do you know?



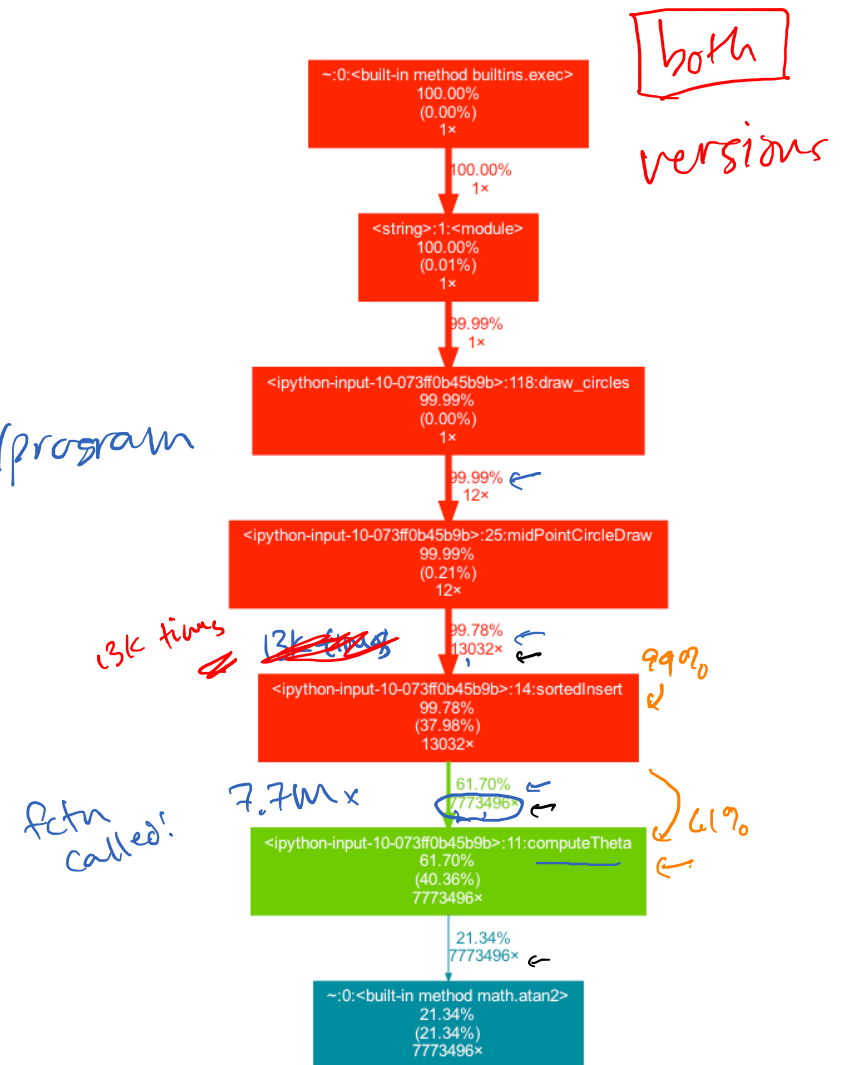
Performance Profiling

- What is profiling?

See where time is spent in application (program)

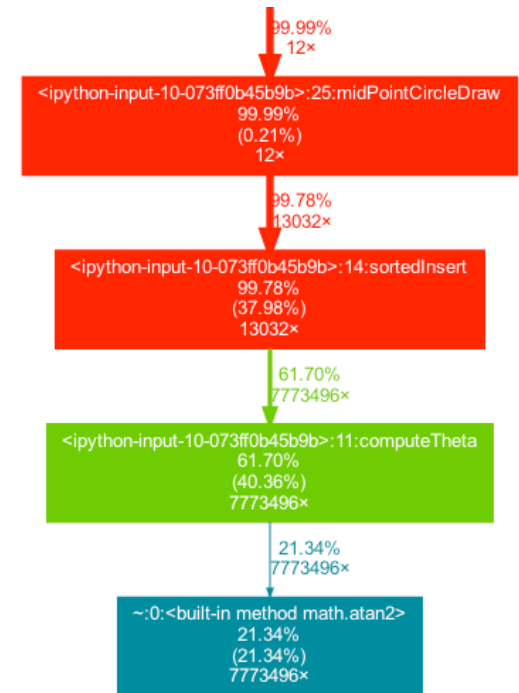
- What function should we be optimizing here?

- How do you know?



Algorithm Tuning

```
def computeTheta(self, x,y, x_centre, y_centre):  
    return math.atan2(x-x_centre, y-y_centre)  
  
def sortedInsert(self, theList, x, y, x_centre, y_centre):  
    for index, value in enumerate(theList):  
        oldTheta = self.computeTheta(value[0],value[1],x_centre,y_centre)  
        newTheta = self.computeTheta(x,y, x_centre, y_centre)  
        if oldTheta > newTheta:  
            theList.insert(index, (x,y))  
            return theList  
    theList.append((x,y))  
    return theList
```



Performance Profiling

```
def computeTheta(self, x,y, x_centre, y_centre):  
    return math.atan2(x-x_centre, y-y_centre)
```

```
def sortedInsert(self, theList, x, y, x_centre, y_centre):
```

```
    for index, value in enumerate(theList):  
        oldTheta = self.computeTheta(value[0],value[1],x_centre,y_centre) value[2]  
        newTheta = self.computeTheta(x,y, x_centre, y_centre) ←  
        if oldTheta > newTheta:  
            theList.insert(index, (x,y))  
        return theList  
    theList.append((x,y, computeTheta(x, y, x_centre, y_centre)))  
    return theList
```

pre-allocate space for points

* remove redundant code

* keep the results of computations around

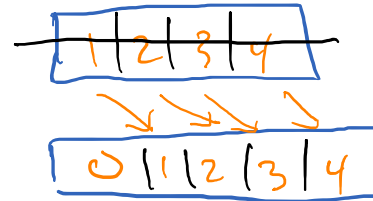
defer +
* bulk process some computations

Data Structures

- When is better here? list or array?
 - Inserting at the beginning?
 - Accessing the element at position N (i.e. `values[n]`)
 - Accessing all elements sequentially?
- What's funny about Python's lists?



Data Structures

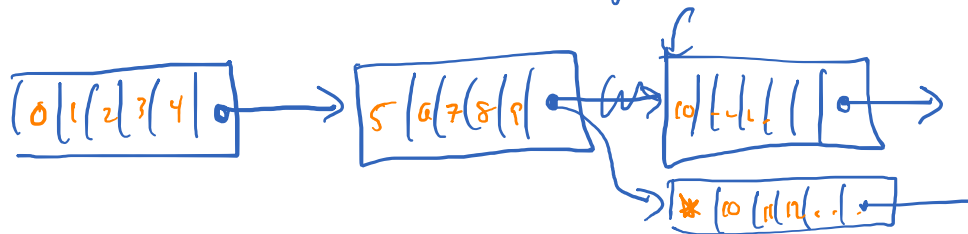


- When is better here? list or array?

- • Inserting at the beginning? *list*
- • Accessing the element at position N (i.e. values[n]) *array*
- • Accessing elements sequentially? *array (slightly)*
(mostly the same)

- What's funny about Python's lists?

→ hybrid array / lists



Bus Interfaces

- AXI4 “Full” vs. AXI4 Lite vs. AXI4 Stream
- Which is easier? Which is faster?
- Which are memory-mapped buses?

Bus Interfaces

P3 -
EMA
(Stream)

P4/P5
Popcut MMIO
(Lite)

P6/P7
PC. DMA
(Full)

- AXI4 “Full” vs. AXI4 Lite vs. AXI4 Stream
- What are the benefits of each?
- Where do we use them?

Benefits -

Full

MMIO
High-Perf.

Lite

MMIO
Simple

Stream

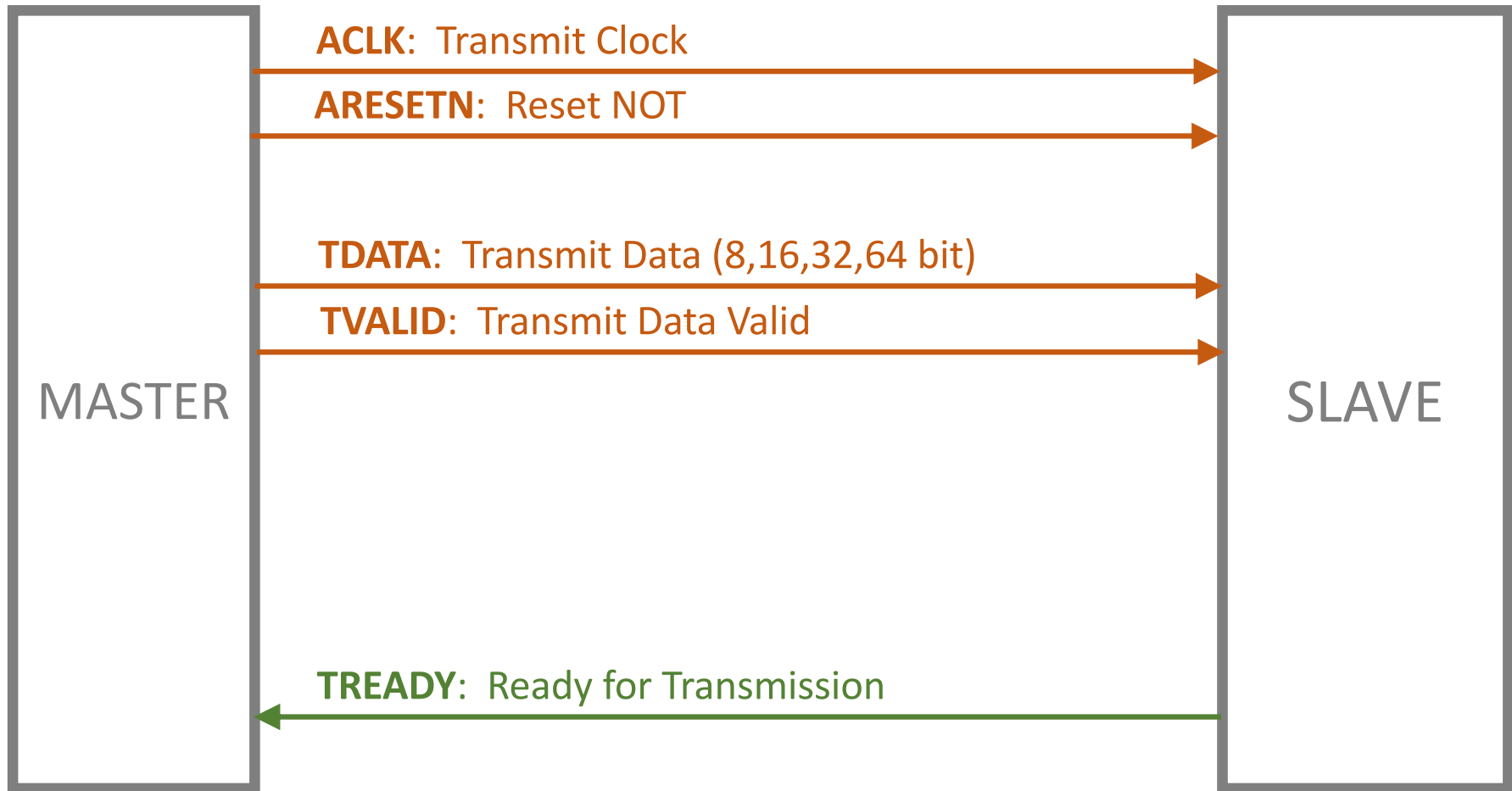
Simple
High Perf.

Draw
backs -

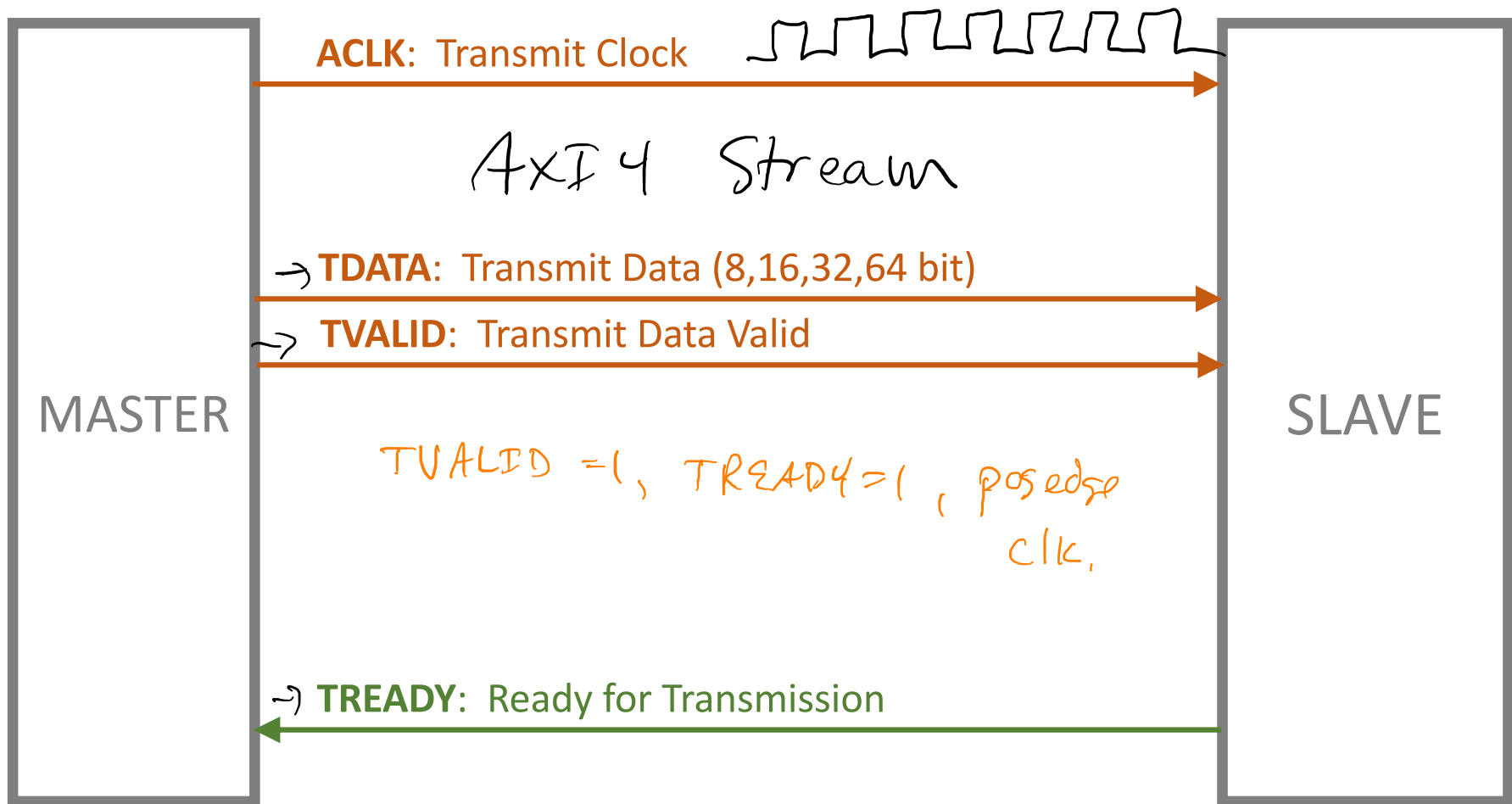
High
Complexity

low perf.

no
MMIO



When is a transmission valid?

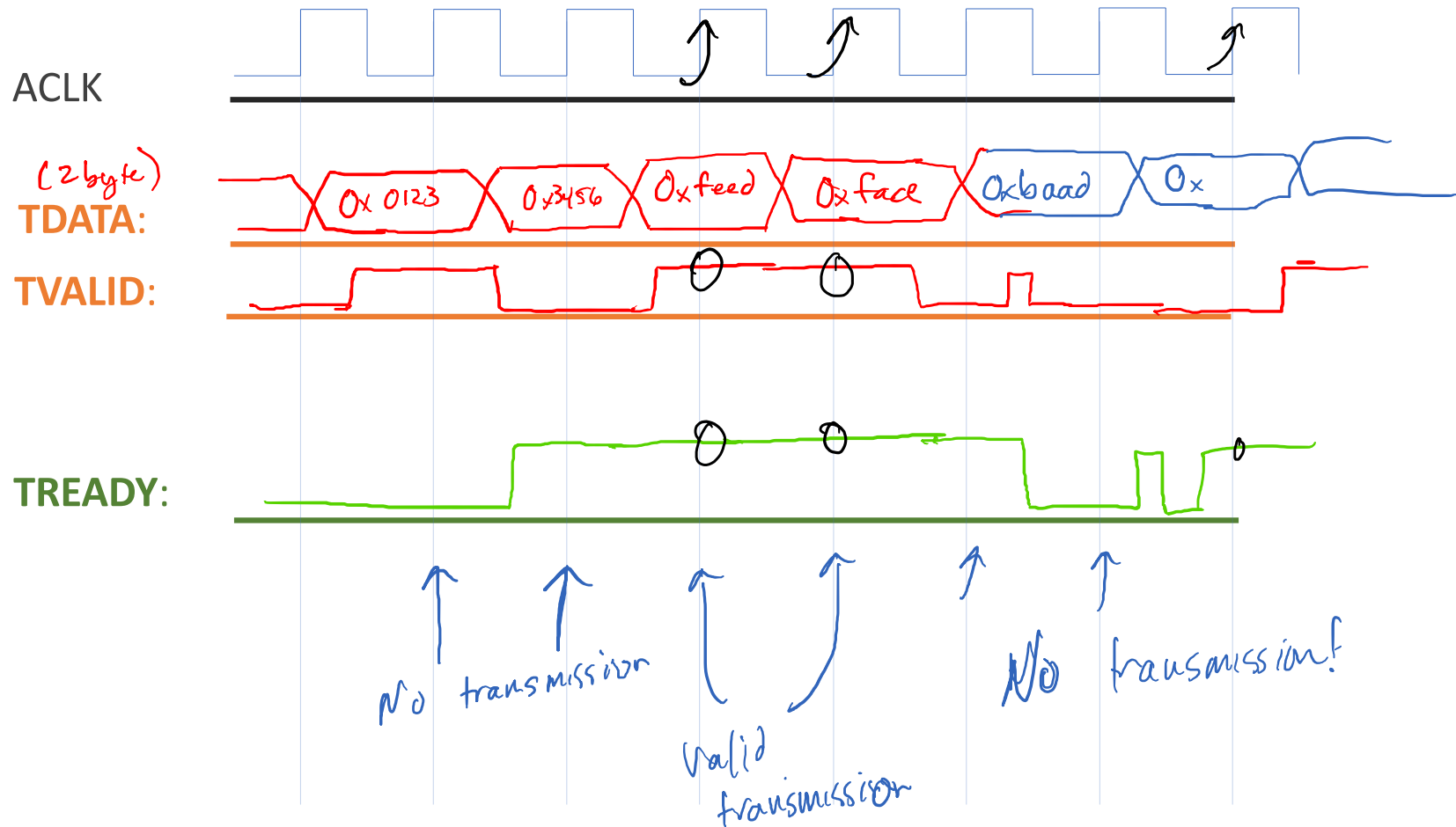


Transferring data on a AXI4-Stream Bus.



Which transmissions are valid?

Transferring data on a AXI4-Stream Bus.



MMIO

- Define MMIO?
- What is MMIO?
- Why do we use it?

MMIO

- Define MMIO? → Memory Mapped Input/Output
 - What is MMIO?
 - Why do we use it?
- maps a hardware piece to memory so CPU can use it.

Hardware block that
"pretends" to be memory

So CPU can interact w/ hardware
using loads / stores.

MMIO Loads

- In ASM?

```
mov r2, 0x40000000 ;  
ldr r3, [r2, 0x144];
```

- In C?

MMIO Loads

← volatile uint32_t * count_reg = vbase_ptr + 0;
*count_reg;

- In ASM?

```
mov r2, 0x40000000 ;  
ldr r3, [*r2, +0x144];
```

load from 0x4000-0140

- In C?

#define ADDR 0x4000-0144

int x = *(volatile uint32_t *) (ADDR)

↑ why? subject to change value randomly

MMIO Loads and Stores

- In ASM?

```
mov r2, 0x40000000 ;  
ldr r3, [r2, 0x144];
```

- In C?

MMIO Loads and Stores

- In ASM?

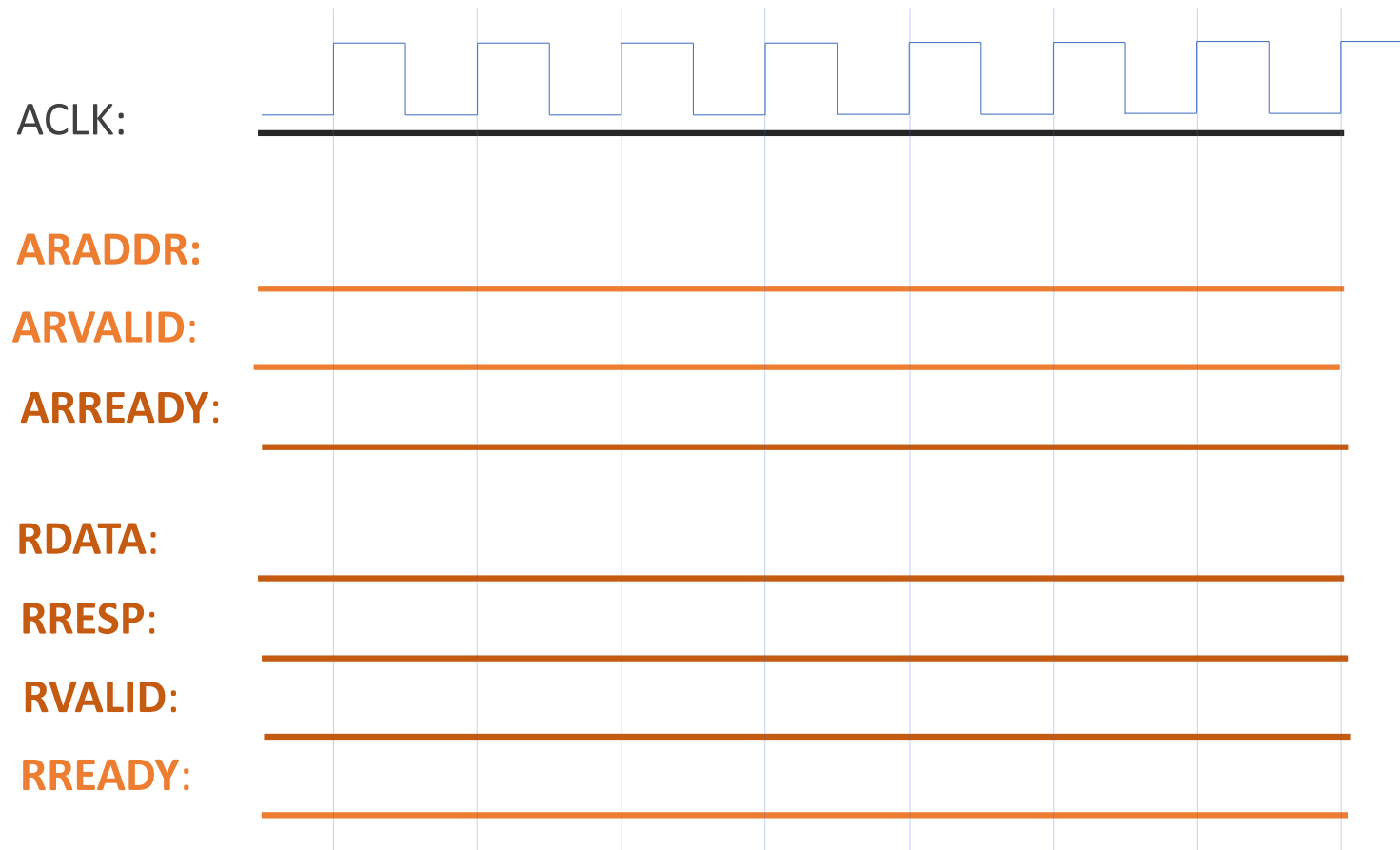
```
mov r2, 0x40000000 ;  
ldr r3, [r2, 0x144];
```

- In C?

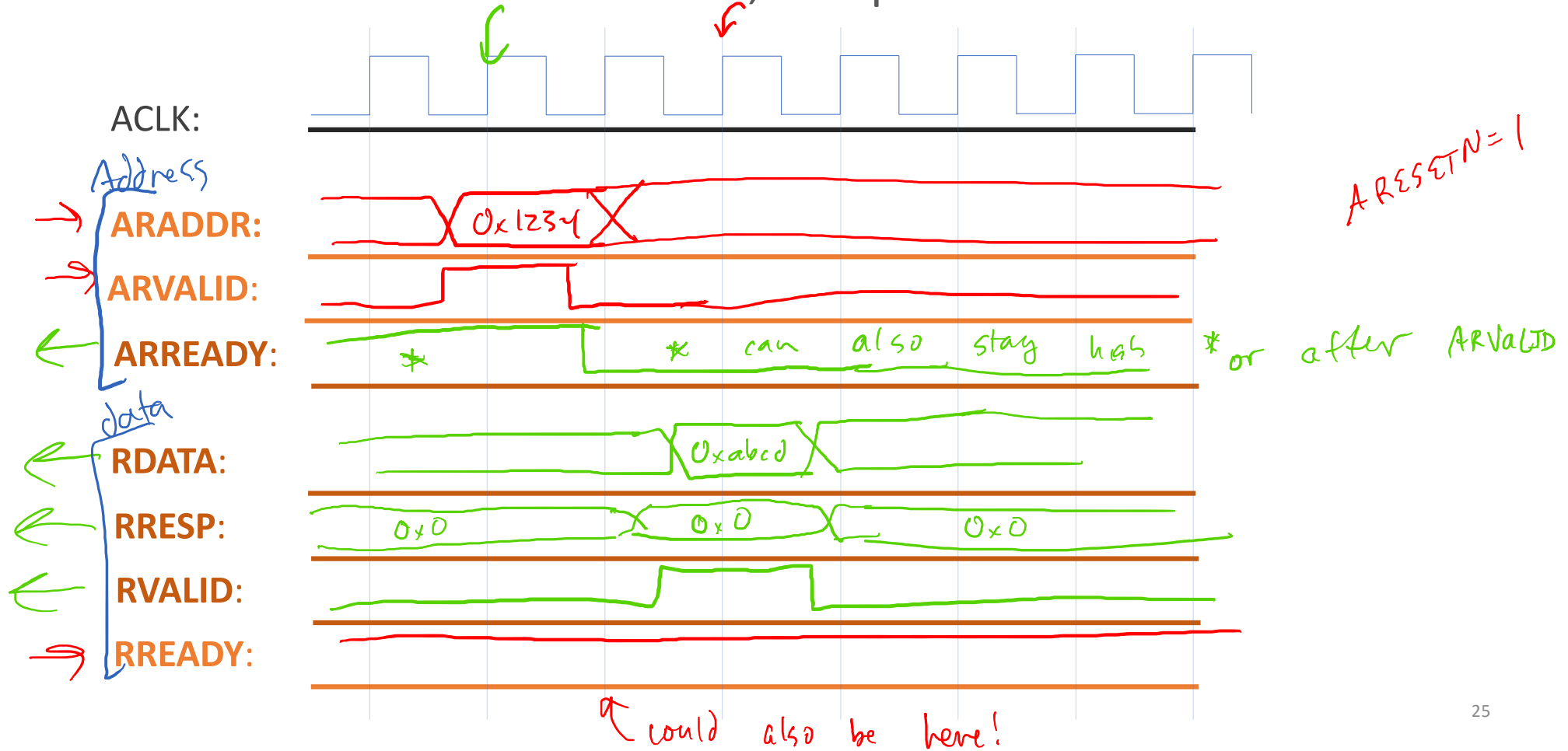
```
uint32_t x = *(volatile uint32_t *)(0x40000144);
```

Store: ~~*~~(volatile uint32_t *) (0x40000144) = 32;

AXI4Lite: Load to address 0x1234, response is 0xabcd



AXI4Lite: Load 0x1234, response: 0xabcd



Linux MMIO?

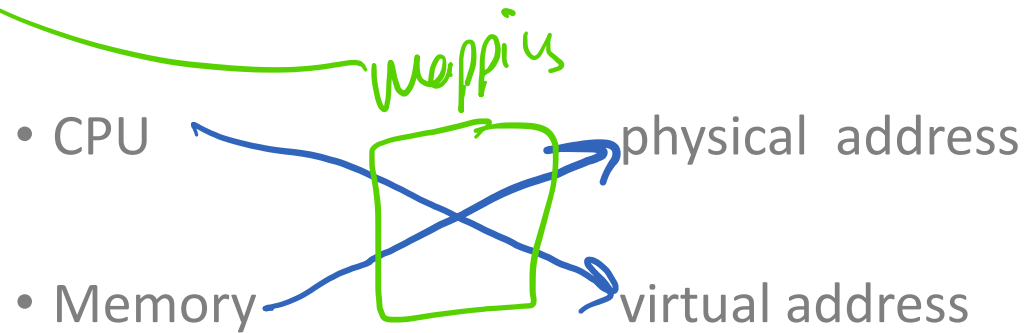
- What's weird about C/MMIO with Linux?

Virtual Memory

- Linux/Hardware “translates” CPU’s memory addresses into real memory addresses.
- | | |
|----------|------------------|
| • CPU | physical address |
| • Memory | virtual address |
| • FPGA | |

Virtual Memory

- Linux/Hardware “translates” CPU’s memory addresses into real memory addresses.

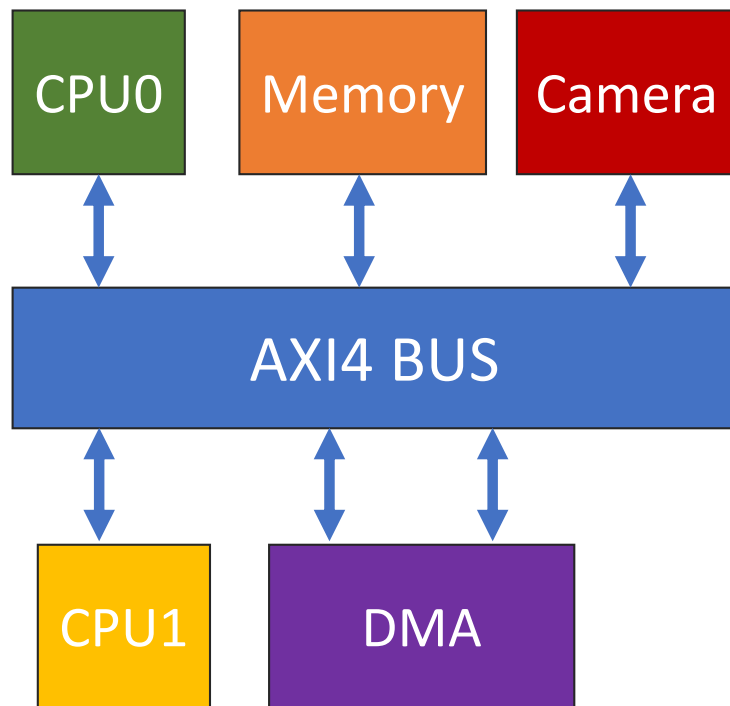


Workaround
virtual → fixed
physical
Address

(0x4000-0000)

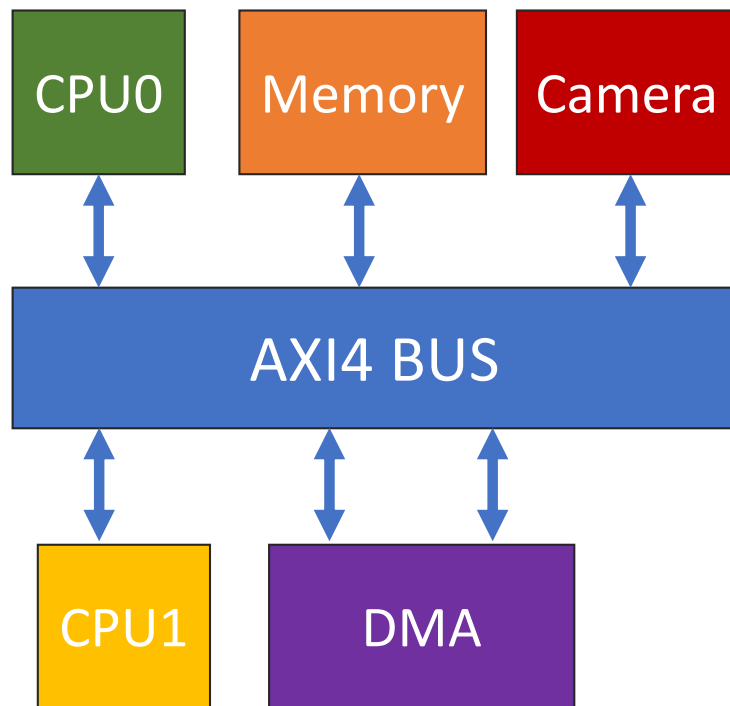
/dev/uio0
+ mmap

DMA



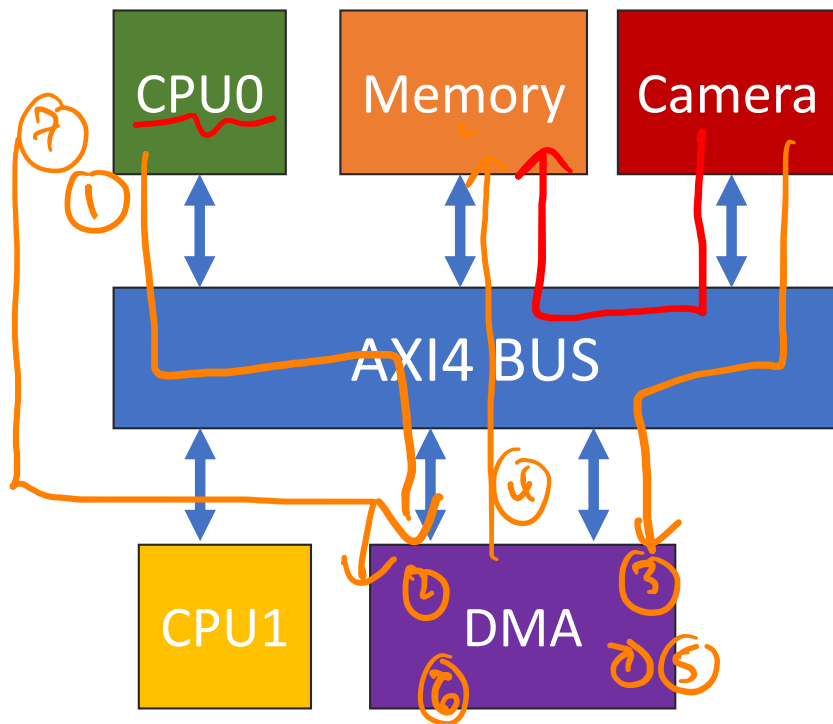
- Define DMA?
- What's the goal of DMA?
- What steps are involved?

DMA



- Define DMA?
- What's the goal of DMA?
- What steps are involved?

DMA



- Define DMA?
- What's the goal of DMA?
- What steps are involved?

→ Direct Memory Access

→ Separate thing CPU used for memory copy

free-up a real CPU, use thing one instead for memory copies

① CPU tells DMA what to copy

② DMA sets "start" bit

③ Load from camera

④ Store to memory

⑤ loop for all memory

⑥ DMA clears "start" bit

⑦ CPU polls for "start" bit clear

DMA Control Design

```
void dma (uint32_t * from,
          uint32_t * to,
          uint32_t size)
{
    register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = from[i]; //load

        to[i] = reg; //store
    }
}
```

- What interfaces do you need?
- How do you start/stop DMA?
- Design a DMA state machine?

DMA Control Design

Start
Start →

```
void dma (uint32_t * from,  
          uint32_t * to, store  
          uint32_t size) store  
{  
    register uint32_t reg;  
  
    for (int i = 0; i < size; ++i){  
  
        reg = from[i]; //load  
  
        to[i] = reg; //store  
  
    }  
}
```

load
Done? ←

- What interfaces do you need? *?* *1 → master* *load/store*
- How do you start/stop DMA? *1 → Slave* *mmio*
- Design a DMA state machine?

All DMA Registers

Control - 0x0400	31-1 Reserved	0 Start
Status - 0x0404	31-1 Reserved	0 Done
Source - 0x0408	31-0 DMA Source Address	
Destination - 0x040C	31-1 DMA Destination Address	
Size - 0x0410	31-16 Reserved	15-0 DMA Transfer Size (in Bytes)

= 1 for done
= 0 for !done

DMA Control

```
void dma (uint32_t * from,
          uint32_t * to,
          uint32_t size)
{
    register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = from[i]; //load

        to[i] = reg; //store
    }
}
```

- AXI4 Master Interface
 - Loads + Stores
- 5 MMIO registers
 - Control (Start)
 - Status (Done)
 - Source (`from`)
 - Destination (`to`)
 - `size` (in **Bytes**)

Using DMA from CPU's side:

```
void dma (uint32_t * from,
          uint32_t * to,
          uint32_t size)
{
```

- 0x0400: Control Register
- 0x0404: Status Register
- 0x0408: Source Address
- 0x040C: Destination Address
- 0x0410: Transfer Size in Bytes

Using DMA from CPU's side:

```
void dma_copy ( uint32_t * src,
                uint32_t * dest,
                uint32_t size){

    *((volatile uint32_t *) (0x0408))=src;
    *((volatile uint32_t *) (0x040C))=dest;
    *((volatile uint32_t *) (0x0410))=size;
    *((volatile uint32_t *) (0x0400))= 0x1; //start

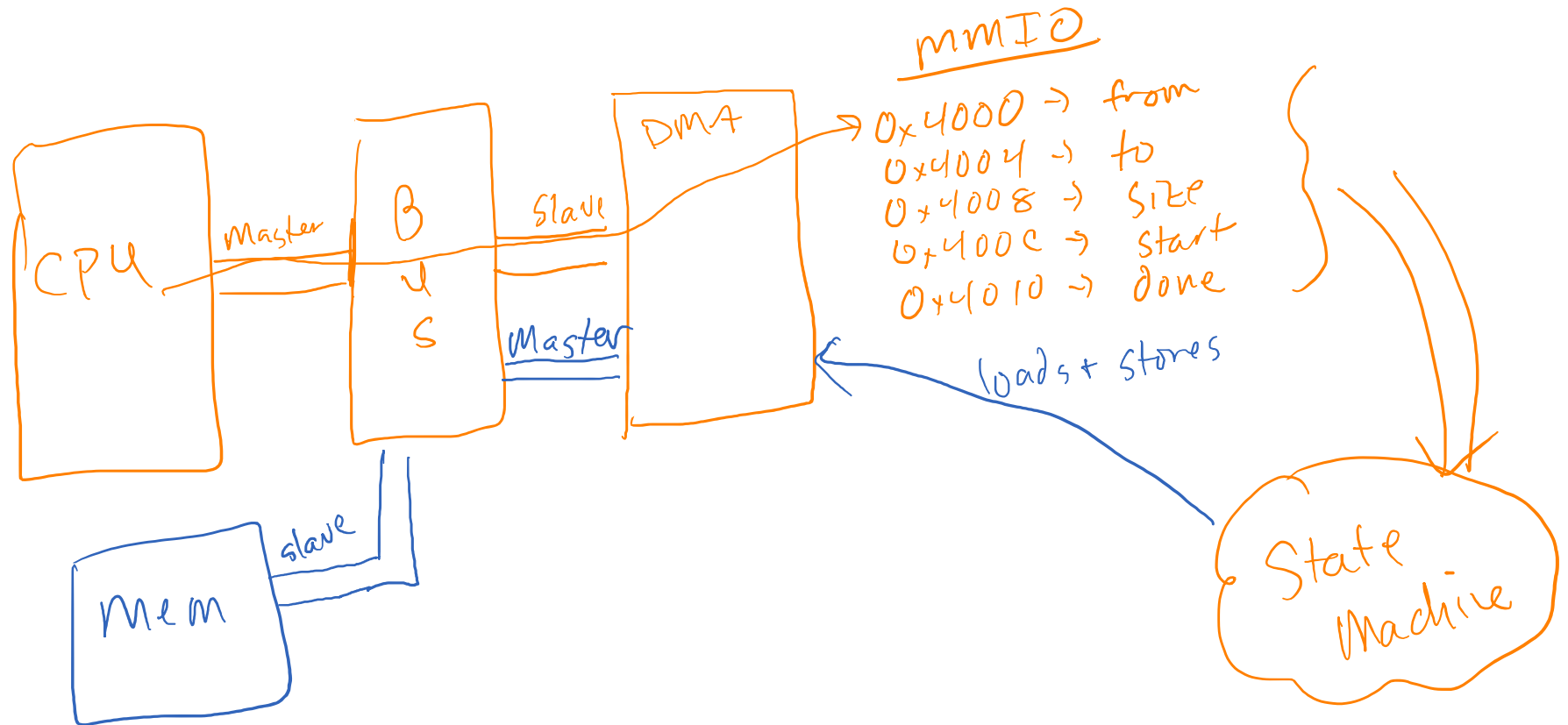
    //spin until copy done
    while( *((volatile uint32_t *) (0x0404)) != 0x1){;}

}
```

0x0400: Control Register
0x0404: Status Register
0x0408: Source Address
0x040C: Destination Address
0x0410: Transfer Size in Bytes

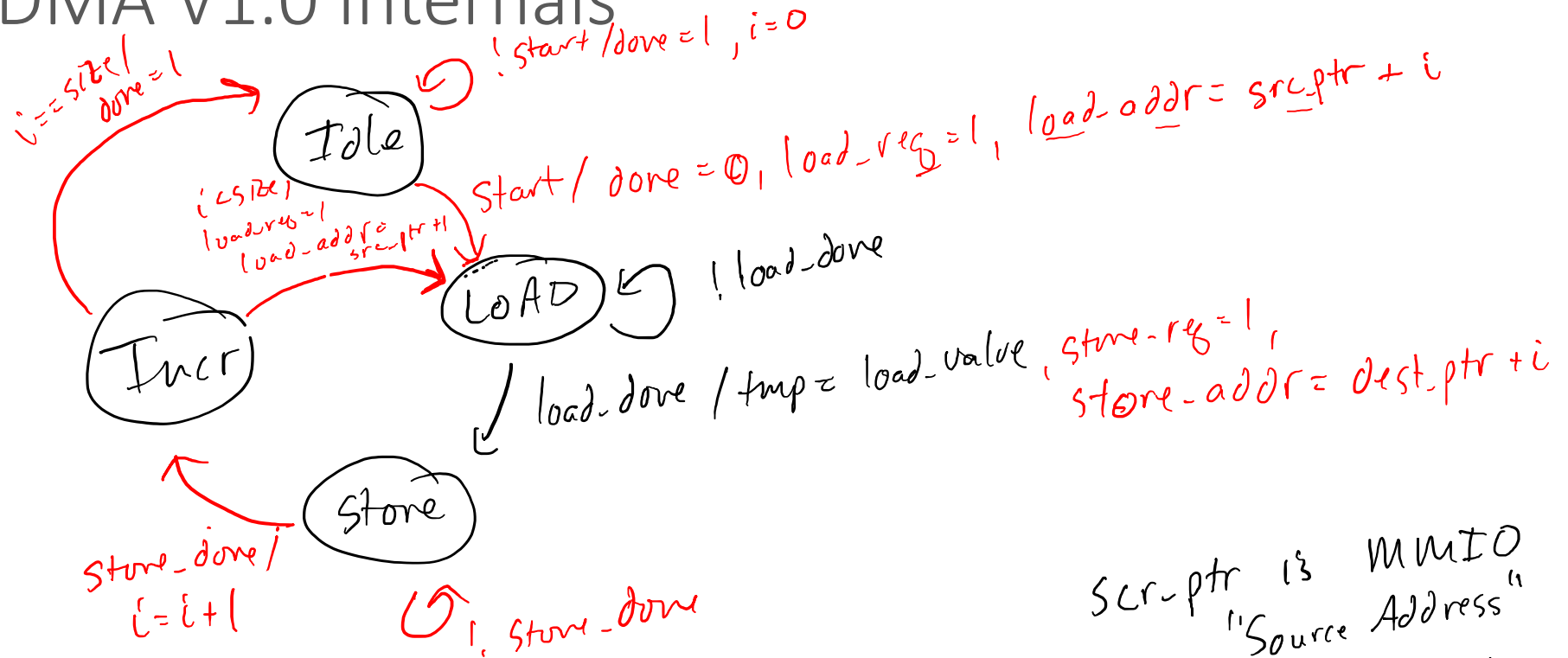
*future exam question: Build own DMA ctrlr.

DMA Control Interface



DMA V1.0 Internals

tmp = temporary Value



src_ptr is MMIO "Source Address"
 dest_ptr is MMIO "Destination Address"

Pipelining

- Define Latency
- Define Throughput

Latency vs. Throughput

- **Latency:** How long does an individual operation take to complete?

individual mult: 8 cycles

- **Throughput:** How many operations can you complete per second (or per cycle)?

Average output per cycles:

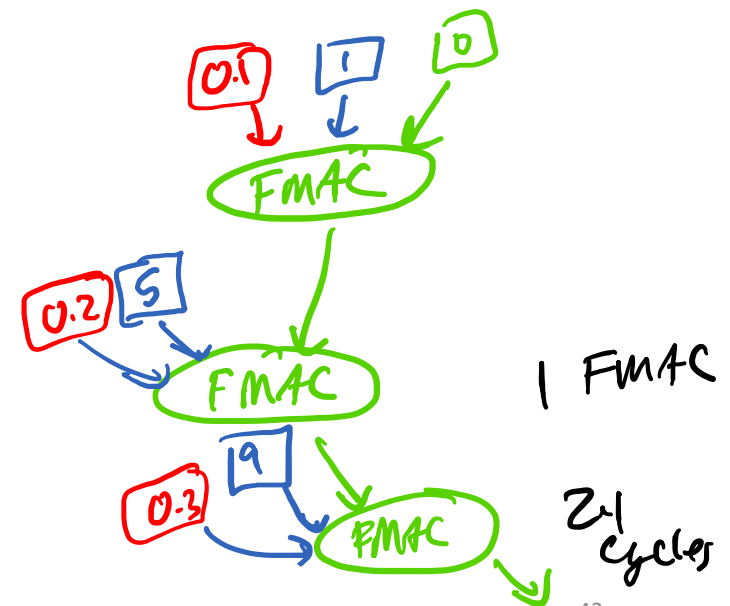
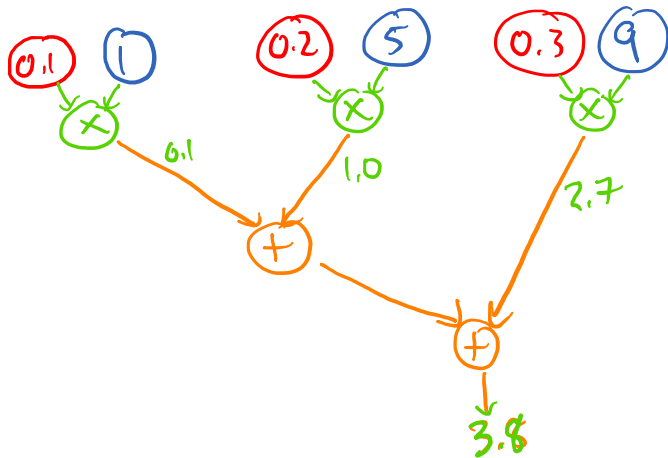
~ 1 mult / cycle

Build a dependence graph for this computation

$$\begin{bmatrix} 0.1 & 0.2 & 0.3 \end{bmatrix} \begin{bmatrix} 1 \\ 5 \\ 9 \end{bmatrix} = \begin{bmatrix} 3.8 \end{bmatrix}$$

How many cycles should this take?

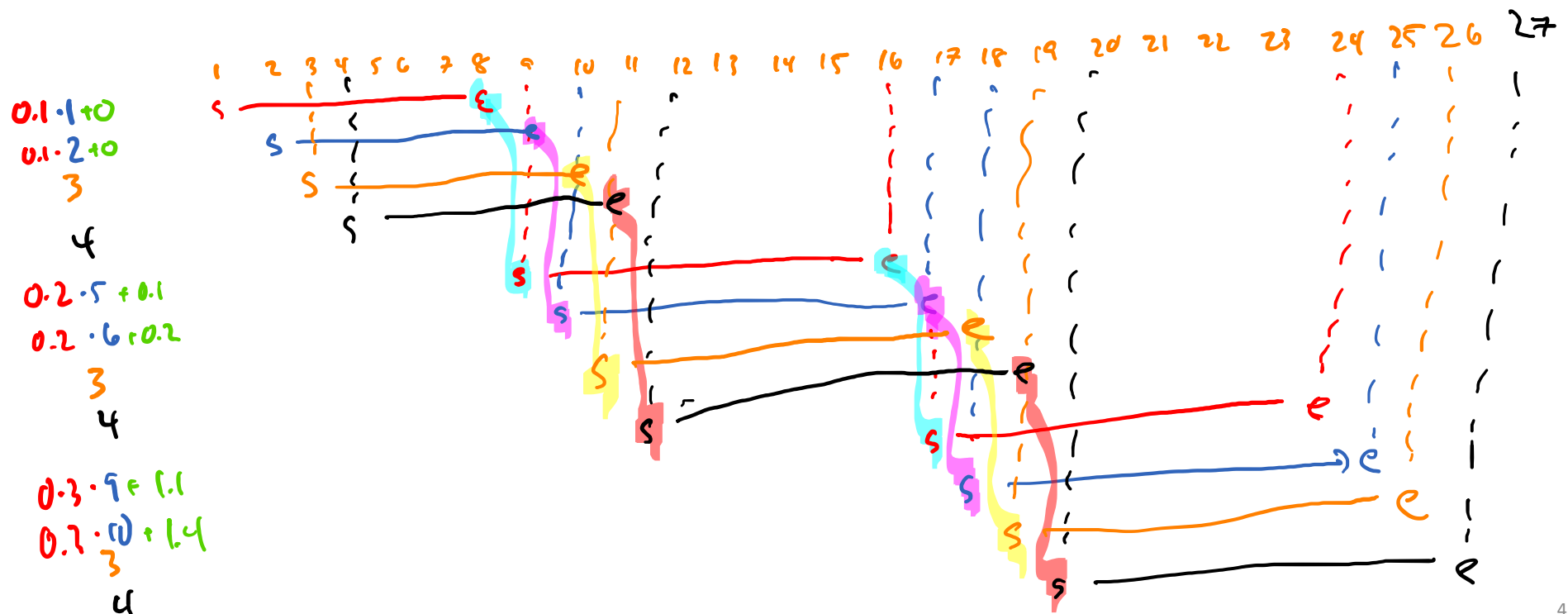
$$\begin{bmatrix} 0.1 & 0.2 & 0.3 \end{bmatrix} \begin{bmatrix} 1 \\ 5 \\ 9 \end{bmatrix} = \begin{bmatrix} 3.8 \end{bmatrix}$$



Build a FMAC timeline for this computation

$$\begin{bmatrix} 0.1 & 0.2 & 0.3 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix} = \begin{bmatrix} 3.8 & 4.4 & 5 & 5.6 \end{bmatrix}$$

Build a FMAC timeline for this computation



Exam Review

Engr 315: Hardware / Software Codesign

Andrew Lukefahr

Indiana University

